# Timing-Sensitive Non-Interference for Interrupt Handling, Compositionally and Mechanically

Willard Rafnsson
IT University of Copenhagen, Denmark
wilr@itu.dk

Dawit Tirore
IT University of Copenhagen, Denmark
dati@itu.dk

***Abstract*—The security of a computer system rests on its operating system, which ties software to hardware and decides which process runs, when, and on whose behalf. A key part of this is interrupt handling. Servicing an interrupt takes time away from whichever process was scheduled. If a public process keeps issuing requests whose results an attacker can see, the handler shows up as a gap in that stream, and the attacker learns that the interrupt occurred. Such timing leaks through interrupt handling arise in several operating systems [TODO: examples, with citations]. Leaks of this kind are studied in information flow control, where their absence is stated as non-interference. In this paper we use the process calculus of Rafnsson et al. to model interrupt handling as a process, and prove in Rocq that it is interfering: a secret disk interrupt is observable through the schedule alone. We then model the corresponding non-interfering variant, obtained by altering how interrupts are processed: they are serviced only inside publicly timed slices, and unused slice time is filled with a dummy handler. We prove it timing-sensitive non-interfering for any scheduler and user processes that are themselves non-interfering. The implementation of the modified interrupt processing is carried out in separate work [TODO: cite X].**

***Index Terms*—non-interference, timing channels, interrupts, process calculi, compositional verification, Rocq**

## I. Introduction

The security of a computer system rests on its operating system. The operating system ties software to hardware: it owns the devices, decides which process runs and when, and stands between processes that must not learn about each other. Whatever an operating system lets one process observe about another is a potential leak, and that includes *when* things happen, not only what values are computed.

A key part of this mediation is interrupt handling. Interrupts are how an operating system learns that something happened: a disk transfer completed, a packet arrived, a timer expired. Some of these events are secret. Whether a particular process just received data from the disk may reveal what it is doing, so an operating system that keeps such an event secret has to hide that it happened at all.

Ordinary interrupt handling does not hide it. As soon as the interrupt can be serviced, its handler runs, displacing whatever process was scheduled; the handler takes a few steps, and then the displaced process resumes. Suppose a public process keeps issuing requests whose results an attacker can see. While the handler runs, those requests stop, and the attacker sees a gap in the stream exactly as long as the handler (Fig. 1). The interrupt has leaked through the *schedule*. Timing leaks through interrupt handling of this kind arise in several operating systems [TODO: examples, with citations].

| step | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| no disk interrupt | `Get` | `Get` | `Get` | `Get` |
| disk interrupt before step 1 | `Get` | — | — | `Get` |

Fig. 1. What the attacker sees of the public process under ordinary interrupt handling, with handlers of two steps. The two runs differ only in whether a disk interrupt arrives, which the attacker may not learn. The interrupt sets a pending bit; step 1 was already scheduled, so the disk handler runs at steps 2 and 3, and the public process does not run then. The mechanised counterexample (Theorem 2) uses steps 1 and 2.

This is the example we follow throughout the paper. The secret is *whether a disk interrupt occurred*. A public process issues requests whose results the attacker sees, and a private process, the legitimate consumer of the disk event, reacts to it. The attacker may learn nothing about the disk interrupt. Sections III to V model this system, exhibit the leak, and remove it.

Leaks of this kind are the subject of information flow control [1], where the absence of a leak is stated as non-interference [2]: varying what the attacker may not see must not change what it does see. For timing leaks, "what it sees" must include when each output appears [3].

In this paper we use the process calculus of Rafnsson, Jia and Bauer [4] to state and prove such properties of interrupt handling. Its notion of non-interference is timing-sensitive, and its non-interference theorems compose: if the parts of a process are non-interfering at suitable equivalences, so is the whole. We model an operating system with interrupt handling as one closed process in this calculus. Inside it, the two user processes (one public, one private), the scheduler and one handler per kind of interrupt run as concurrent processes in a pool, and a global state, holding the interrupt controller, decides at each step which of them runs. Interrupts arrive as inputs to the whole system, and at each step the pool emits the output of

the process that ran. We prove in Rocq[1] that ordinary interrupt handling is interfering (`model_immediate_not_NI`): the leak of Fig. 1 is a counterexample.

We then model the corresponding non-interfering variant, obtained by altering how interrupts are processed. The idea is to take away a secret interrupt's power to choose *when* its handler runs. Interrupts are serviced only inside time slices opened by the public timer, every handler run takes the same number of steps, and any part of a slice that no real interrupt claims is filled with a dummy handler, the NOP handler. A secret handler run then replaces a NOP run instead of displacing a user process, and the public schedule is the same whether or not the interrupt occurred. We prove this variant non-interfering (`model_sliced_NI`), even to an attacker who sees, at every step, which process in the pool ran, the scheduler and the handlers included. Both variants are instances of a single generic model and differ only in their *interrupt policy*: the initial state and how handler completion is serviced. The result is general in two ways. It holds for every scheduler and every pair of user processes that are themselves non-interfering. And it holds for every choice of the two numbers that fix the timing of the policy: the number of steps one handler run takes, and the number of handler runs that fit in one time slice.

The model fixes *what* the modified interrupt processing must do; how to implement it in an operating system is carried out in separate work [TODO: cite X].

*a) Contributions:*

- A mechanised, timing-sensitive non-interference proof for interrupt handling (Sections IV and V), and a mechanised counterexample for ordinary interrupt handling (Section IV-A). The proof is compositional: it applies one generic theorem per constructor of the model, and leaves one substantial obligation specific to interrupt handling (Section V-E).
- Requirements, forced by the proof, on any implementation of interrupt handling that is to be non-interfering (Section V). The arrival of a secret interrupt may only record that it is pending; every decision it influences, such as starting its handler, must be deferred to a point fixed by public time. Whether a secret handler ran must be indistinguishable from whether a dummy handler ran, so the time reserved for handlers must be fixed in advance and always used up. And the interrupt controller must be split by secrecy: which interrupts are pending is secret, while the masks that decide what runs next may depend only on public information. The implementation in [TODO: cite X] follows these requirements.
- A mechanisation of the calculus and its composition theorems. Our definitions depart from the original in one essential place: non-interference is stated over finite traces, which makes every proof an induction. We prove the definition equivalent to the original coinductive one, so our main results also hold in the original's terms (Section VII).

Section II recalls the calculus and the definition of non-interference. Section III presents the generic model and Section IV the two interrupt policies. Section V gives the proof, Section VI states precisely how general it is, and Section VII describes the mechanisation. Sections VIII and IX discuss related work and limitations.

## II. Background: Composable Timing-Sensitive Non-Interference

This section recalls the calculus and definitions of Rafnsson et al. [4], in the form we mechanise. The one essential difference is that non-interference is stated over finite traces; Section VII-A proves it equivalent to the original.

### A. Processes, traces and time

A process $p : \mathit{Proc}\ I\ O$ has an input type $I$ and an output type $O$. It moves in one of two ways, by receiving an input, $p \overset{i}{\rightsquigarrow} p'$, or by emitting an output, $p \xrightarrow{o} p'$, in any order. Processes are built from seven constructors, whose reduction rules are given in Fig. 2. The operating system of Section III is a single closed term over them.

`out o` ignores its input and always emits `o`. `map f g p` translates inputs by `f` on the way into `p` and outputs by `g` on the way out. `sta f g v p` keeps a state cell `v`: an input `i` updates it to `f i v`, and `p` receives the new state together with `i`; an output `o` of `p` updates it to `g o v`, and the new state is emitted together with `o`. `swi b p` switches `p` on and off. While the phase `b` is `false` it emits `None` and `p` does not move; while it is `true` it emits `Some o` for an output `(c,o)` of `p`. The phase is toggled by the tag `c` and by the bit carried on each input. `par p1 p2` gives every input to both parts and emits both outputs as a pair. `loop p` feeds each output of `p` back to it as an input. `maybe p` makes the input optional: `None` leaves `p` unchanged.

Reduction is deterministic, and a process never blocks.

Time is counted in output steps: every output step is one tick of a clock the observer shares, and on every tick a process emits a value (in our models, a component that did not run emits `None`). Input steps take no time. This makes the setting timing-sensitive: a process that emits `o` one step late emits something else, say `None`, at the original step, so a delay is observed as a change of value.

### B. Characterised equivalences and non-interference

Security is stated relative to a lattice of levels $\ell$. Everything in this paper can be read with the two-point lattice $\mathsf{L} \sqsubseteq \mathsf{H}$: the attacker $\mathsf{L}$, and $\mathsf{H}$, who sees everything (Section VI discusses arbitrary lattices). Each input and output type carries a *characterised equivalence* (`cEquiv`) $\doteq$, which says, for each level $\ell$:

- $x \doteq_\ell y$: an observer at $\ell$ cannot tell $x$ from $y$;
- $x \doteq_\ell \bullet$: an observer at $\ell$ does not see $x$ at all.

[1]The mechanisation is available at https://github.com/Tirore96/NonInterference.

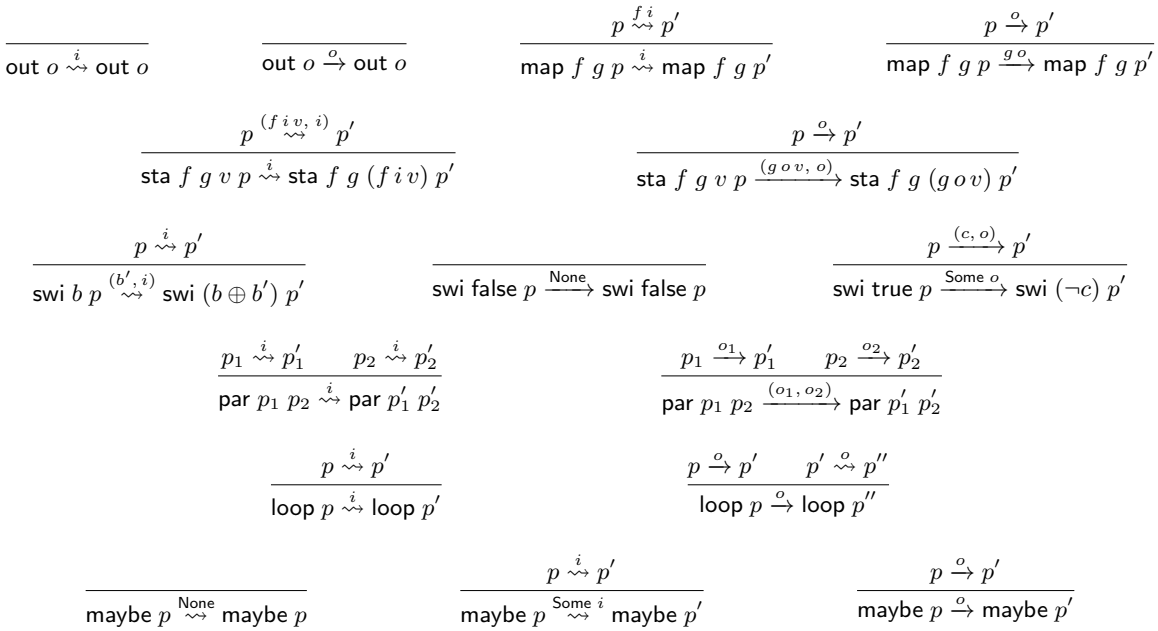


Fig. 2. Reduction rules (`reduceI`, `reduce0`). Wavy arrows are input steps, straight arrows output steps; $\oplus$ is exclusive or.

We keep the notation of Rafnsson et al.; in the mechanisation $x \doteq_\ell \bullet$ is a predicate on $x$, which changes nothing (Section VII-A). $\doteq_\ell$ is an equivalence, and the unobservable values form one of its classes:

$$x \doteq_\ell \bullet \;\Rightarrow\; \forall y.\, (y \doteq_\ell \bullet \Leftrightarrow x \doteq_\ell y).$$

So at each level the type is partitioned into classes the observer cannot tell apart, and one of them, the *distinguished class*, holds the values the observer does not see. It is empty when the level hides nothing. We write $\overset{I}{\doteq}$ and $\overset{O}{\doteq}$ for the equivalences on inputs and outputs.

A trace $t$ is a finite word of input effects $?i$ and output effects $!o$. We write $\epsilon$ for the empty trace and "." for concatenation. A process $p$ *performs* $t$ at $\ell$, written $p \overset{t}{\dashrightarrow}_\ell$, if it has a run with the effects of $t$, outputs compared up to $\overset{O}{\doteq}_\ell$ (`Trace`):

$$\frac{}{p \overset{\epsilon}{\dashrightarrow}_\ell} \qquad \frac{p \overset{i}{\rightsquigarrow} p' \qquad p' \overset{t}{\dashrightarrow}_\ell}{p \overset{?i.t}{\dashrightarrow}_\ell}$$

$$\frac{p \xrightarrow{o'} p' \qquad o' \overset{O}{\doteq}_\ell o \qquad p' \overset{t}{\dashrightarrow}_\ell}{p \overset{!o.t}{\dashrightarrow}_\ell}$$

**Definition 1** (Non-interference, `NI`)**.** $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq})$ iff for all $\ell$, $t$, $t'$, $i$ and $i'$:

$$i \overset{I}{\doteq}_\ell i' \;\Rightarrow\; \left(p \overset{t.?i.t'}{\dashrightarrow}_\ell \Rightarrow p \overset{t.?i'.t'}{\dashrightarrow}_\ell\right),$$
$$i \overset{I}{\doteq}_\ell \bullet \;\Rightarrow\; \left(p \overset{t.t'}{\dashrightarrow}_\ell \Leftrightarrow p \overset{t.?i.t'}{\dashrightarrow}_\ell\right).$$

The first clause (substitution) says that related inputs are interchangeable. The second says that an input the observer does not see may be inserted into a trace (insertion, $\Rightarrow$) or deleted from it (deletion, $\Leftarrow$) without the observer noticing. The set of traces visible at $\ell$ is thus unchanged by anything the observer at $\ell$ may not see.

### C. Equivalences

Two equivalences do most of the work. Under `public_equiv` nothing is unobservable and $x \doteq_\ell y$ iff $x = y$, at every level. Under `private_equiv` every value is unobservable at $\mathsf{L}$, where all values are related, and the equivalence is `public_equiv` at $\mathsf{H}$.

Composite equivalences are built from these. $\mathsf{eqpair}(\overset{1}{\doteq}, \overset{2}{\doteq})$ (`eqpair`) relates pairs componentwise, and no pair is unobservable. Its variants `eqpair_L`, `eqpair_R` and `eqpair_LR` make a pair unobservable when its left, its right, or both components are. $\mathsf{eqsum}(\overset{1}{\doteq}, \overset{2}{\doteq})$ (`eqsum`) relates a sum tag by tag and never relates an `inl` to an `inr`, so the tag stays public even when the payload is not; `eqsum_L` makes an `inl` unobservable exactly when its payload is.

Options are related by $\mathsf{eqmaybe}(L, \doteq)$ (`eqmaybe`), where $L$ is the set of levels that can see `None`. It is the least equivalence with

$$\mathsf{Some}\ v \doteq_\ell \mathsf{Some}\ v' \iff v \doteq_\ell v',$$
$$\mathsf{Some}\ v \doteq_\ell \bullet \iff v \doteq_\ell \bullet,$$
$$\mathsf{None} \doteq_\ell \bullet \iff \ell \notin L.$$

So `None` is secret exactly for the levels outside $L$, and, by the characterisation law, `Some v` and `None` are related at $\ell$

TABLE I
THE INSTANCES OF eqmaybe$(L, \doteq)$, BY WHO CAN SEE NONE. TO THE ATTACKER, EQMAYBE_NONBOT AND EQMAYBE_NOBODY AGREE.

| equivalence | $L$ | L sees | H sees |
|---|---|---|---|
| eqmaybe_all | all levels | yes | yes |
| eqmaybe_nonbot | $\{\ell \mid \ell \neq \mathsf{L}\}$ | no | yes |
| eqmaybe_nobody | $\emptyset$ | no | no |
| eqmaybe_aware$(\overset{O}{\doteq}, \overset{B}{\doteq})$ | $\mathsf{A}(\mathsf{true}, \overset{B}{\doteq})$ | if aware | if aware |

exactly when both are unobservable there. The four instances we use are named after who sees None (Table I).

### D. Compositional theorems

The calculus comes with one non-interference theorem per constructor. Each derives the equivalences of the composite from those of its parts, which is what gives the equivalences of a large model their shape. Their side conditions are stated with the following predicates on functions and levels:

$$f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq}) \iff \forall \ell, i, i'.\ i \overset{I}{\doteq}_\ell i' \Rightarrow f\, i \overset{O}{\doteq}_\ell f\, i'$$

$$f \in \mathsf{PU}(\overset{I}{\doteq}, \overset{O}{\doteq}) \iff \forall \ell, i.\ i \overset{I}{\doteq}_\ell \bullet \Rightarrow f\, i \overset{O}{\doteq}_\ell \bullet$$

$$f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq}) \iff \forall \ell, i, i', v, v'.\ i \overset{I}{\doteq}_\ell i' \wedge v \overset{V}{\doteq}_\ell v' \Rightarrow f\, i\, v \overset{V}{\doteq}_\ell f\, i'\, v'$$

$$f \in \mathsf{PE}(\overset{I}{\doteq}, \overset{V}{\doteq}) \iff \forall \ell, i, v.\ i \overset{I}{\doteq}_\ell \bullet \Rightarrow f\, i\, v \overset{V}{\doteq}_\ell v$$

$$\ell \in \mathsf{A}(v, \doteq) \iff \forall \hat{v}.\ v \doteq_\ell \hat{v} \Rightarrow v = \hat{v}$$

$$\ell \in \mathsf{O}(p, \doteq) \iff \text{every output of } p \text{ at } \ell \text{ is } \doteq_\ell \bullet$$

A function is non-interfering if it maps related values to related values (f_NI, fv_NI), unobservable-preserving if it maps unobservable values to unobservable ones (f_PU), and equivalence-preserving if an unobservable input does not visibly move the state (f_EP). An observer is *aware* of $v$ (aware) if it can tell $v$ from every other value, and *oblivious* to $p$ (oblivious) if it sees none of $p$'s outputs.

**Theorem 1** (Composition [4])**.** *For all processes, functions and equivalences of matching types:*

1) *(out_NI)* out $o \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq})$.
2) *(map_NI) If* $p \in \mathrm{NI}(\overset{I'}{\doteq}, \overset{O}{\doteq})$, $f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{I'}{\doteq}) \cap \mathsf{PU}(\overset{I}{\doteq}, \overset{I'}{\doteq})$ *and* $g \in \mathrm{NI}(\overset{O}{\doteq}, \overset{O'}{\doteq})$*, then* map $f\ g\ p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O'}{\doteq})$.
3) *(sta_NI) If* $p \in \mathrm{NI}(\mathsf{eqpair_R}(\overset{V}{\doteq}, \overset{I}{\doteq}), \overset{O}{\doteq}), f \in \mathrm{NI}(\overset{I}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq}) \cap \mathsf{PE}(\overset{I}{\doteq}, \overset{V}{\doteq})$ *and* $g \in \mathrm{NI}(\overset{O}{\doteq}, \overset{V}{\doteq}, \overset{V}{\doteq})$*, then* sta $f\ g\ v\ p \in \mathrm{NI}(\overset{I}{\doteq}, \mathsf{eqpair}(\overset{V}{\doteq}, \overset{O}{\doteq}))$.
4) *(swi_NI) If* $p \in \mathrm{NI}(\overset{I}{\doteq}, \mathsf{eqpair_LR}(\overset{B}{\doteq}, \overset{O}{\doteq}))$ *and for every* $\ell$, $\ell \in \mathsf{A}(\mathsf{true}, \overset{B}{\doteq}) \cup \mathsf{O}(\mathsf{swi}\ b\ p, \mathsf{eqmaybe_aware}(\overset{O}{\doteq}, \overset{B}{\doteq}))$, *then* swi $b\ p \in \mathrm{NI}(\mathsf{eqpair_LR}(\overset{B}{\doteq}, \overset{I}{\doteq}), \mathsf{eqmaybe_aware}(\overset{O}{\doteq}, \overset{B}{\doteq}))$.
5) *(par_NI) If* $p_1 \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O_1}{\doteq})$ *and* $p_2 \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O_2}{\doteq})$*, then* par $p_1\ p_2 \in \mathrm{NI}(\overset{I}{\doteq}, \mathsf{eqpair}(\overset{O_1}{\doteq}, \overset{O_2}{\doteq}))$.
6) *(loop_NI) If* $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{I}{\doteq})$*, then* loop $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{I}{\doteq})$.
7) *(maybe_NI) If* $p \in \mathrm{NI}(\overset{I}{\doteq}, \overset{O}{\doteq})$*, then* maybe $p \in \mathrm{NI}(\mathsf{eqmaybe_nobody}(\overset{I}{\doteq}), \overset{O}{\doteq})$.

Two of these carry substantial side conditions: the state update of sta must be non-interfering and equivalence-preserving, and swi requires every observer to be aware of the gate or oblivious to the gated process. These are the two obligations the proof of Section V has to discharge for the model.

## III. AN OPERATING SYSTEM AS ONE PROCESS

Section II gave us a calculus of processes, equivalences that say what an observer at each level sees, and one composition theorem per constructor. We now use the calculus to write an operating system with interrupt handling as a single process, following the example of Fig. 1: the secret is whether a disk interrupt occurred. We first introduce the processes the system runs and the pool in which they run (Section III-A), and what the system receives and emits (Section III-B). We then describe the state (Section III-C) and how each event updates it, ending in the choice of who runs next (Section III-D). The resulting generic model leaves two parts open, which together form the *interrupt policy* (Section III-E); Section IV fills them twice.

### A. The processes and the pool

The system runs six processes: three interrupt handlers, a scheduler, and two user processes. The timer, disk and NOP handlers are copies of one process: a handler run takes exactly runtime steps, and on its last step the handler emits Notify to announce completion. The public user process issues requests (Get) whose results the attacker sees. The private user process takes the disk handler's output as input, and emits Sys when it has been notified of a disk event and Idle otherwise. When the scheduler runs, it proposes which user process runs next.

The six processes run side by side in a *pool* (Fig. 3), and only one of them takes a step at a time. Each sits behind a switch, controlled by the current process identifier (pid): a pid names one slot, whose process takes one step, while every other slot emits None without moving. The pid is held in a global state, which a state cell and a feedback loop thread around the pool.

### B. Inputs and outputs

The system receives interrupts. Its input type T_in has two values, the timer interrupt and the disk interrupt. An interrupt goes to the state (Section III-C), where it marks itself pending in the interrupt controller.

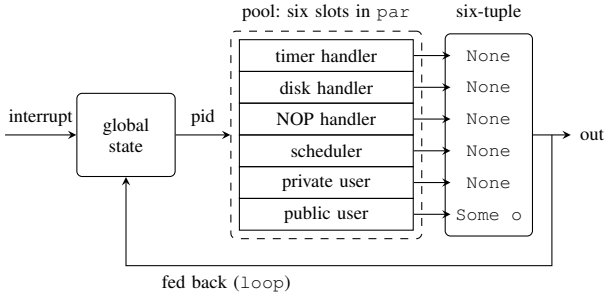



Fig. 3. The model. An interrupt only updates the global state. The state's pid selects one slot, which takes a step and emits `Some o`; every other slot emits `None`. Because the slots are composed with `par`, their outputs form a six-tuple, which is emitted and fed back to update the state.

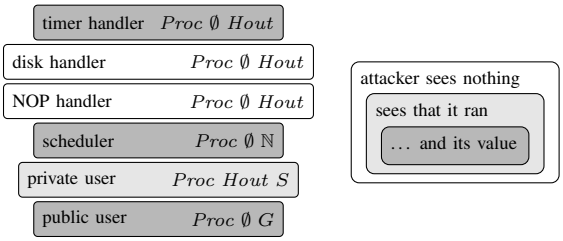



Fig. 4. The six processes, with their types, shaded by what the attacker learns of each (key on the right). $Hout = \{\texttt{Notify}, \texttt{Nothing}\}$ is the handler output, which the private user reads from the disk handler; $S = \{\texttt{Sys}, \texttt{Idle}\}$, $G = \{\texttt{Get}\}$, and the scheduler emits a pid.

The disk interrupt is the secret. The input equivalence `in_equiv` makes it, and only it, unobservable to the attacker; otherwise interrupts are related only when equal:

$$ir \doteq_\ell \bullet \iff ir = \mathsf{Disk} \wedge \ell = \mathsf{L},$$
$$ir \doteq_\ell ir' \iff ir = ir' \vee (ir \doteq_\ell \bullet \wedge ir' \doteq_\ell \bullet).$$

By the rule for `par` (Fig. 2), the pool emits the outputs of all its slots together. So the system emits a six-tuple of options, the output type `T_out`, with exactly one `Some` on each step. The output therefore records not only what was computed but also *who ran*: the schedule is part of what the system emits. Figure 4 shows what we intend to hide from the attacker in each position. The exact output equivalence, and why the proof forces its shape, come in Section V-A. Each output, like each input, then updates the state.

### C. The state

The global state (`state_type`) is a nest of pairs:

```
state_type = (cur_pid × prev_pid)
           × (re_sched × (ir_count × ic))
cur_pid  = Sum Bool Nat
prev_pid = Option Nat
re_sched = Bool
ir_count = Option Nat
ic       = ir_bits × (ir_bits × ir_bits)
ir_bits  = Bool × Bool    (* pending, mask *)
```

- The *current pid* names the slot that runs next. The two secret handlers sit on the `inl` side, `inl true` for the disk handler and `inl false` for the NOP handler, and every other process on the `inr` side, numbered from 0: timer handler, scheduler, private user, public user. The system starts in the public user (`initial_pid`).
- The *previous pid* is the user process a handler interrupted, to which control returns when the handler finishes.
- The *reschedule flag* `re_sched` sends control to the scheduler instead, when a handler finishes.
- The *slice counter* `ir_count` is used only by the sliced policy of Section IV-B; `None` disables it.
- The *interrupt controller* `ic` holds a pending bit and a mask bit for each handler. Its entries are indexed by `irq`, an optional interrupt: `timer_irq` and `disk_irq` for the two interrupts, and `nop_irq`, which is `None`, for the NOP handler. No interrupt raises the NOP entry; the sliced policy sets it pending itself, to use the NOP handler as filler.

The initial state is not fixed here: it is part of the interrupt policy (Section III-E).

### D. Who runs next

The state is updated once per event, by

```
state_step service_policy restore_invariant =
  step_sum set_pending
    (initiate_next restore_invariant
     o service_policy o check_scheduler)
```

`step_sum` applies `set_pending` to an input, and the composition, right to left, to a pool output. On an interrupt, `set_pending` sets that interrupt's pending bit. On a pool output, `check_scheduler` first adopts the pid the scheduler proposed, if the scheduler ran. The next stage, `service_policy`, is left open here: it is half of the interrupt policy (Section III-E). Last, `initiate_next` computes the next pid.

A handler can be started when its controller entry is pending and not masked; if several can, the timer goes first, then the disk, then NOP. Handlers cannot be interrupted: starting one sets all masks. `initiate_next` chooses by a priority rule:

1) if a handler is running, it continues;
2) otherwise, if a handler can be started, it starts;
3) otherwise, if a handler has just finished, control returns to the user process it interrupted, or to the scheduler if a reschedule was requested;
4) otherwise, the current process continues.

Just before the choice, `initiate_next` applies `restore_invariant`, a hook used only by the proof (Section III-E). Appendix A gives the code of `initiate_next`.

This is enough to see the leak of Fig. 1. With the disk unmasked, a disk interrupt sets its pending bit. At the next output, rule 2 applies and the disk handler starts at once, and for the `runtime` steps of the handler the public user's slot emits `None`.

### E. The generic model

Together, these parts form the generic model

```
model runtime init_policy service_policy
      restore_invariant p_pub p_priv p_sched
  : Proc T_in T_out
```

(model, written out in Appendix A). Its first argument, runtime, is the handler length, and its last three are the scheduler and the two user processes. Everything else in Sections III-A to III-D is fixed, except two parts, marked by the suffix _policy:

- init_policy, the initial state, a value of the type of Section III-C. Its masks decide which interrupts can be serviced before anything has happened.
- service_policy, the open stage of Section III-D. It is the only stage that sees a handler announce completion, and so the only place masks can be reopened: it decides when the next handler becomes eligible.

Together they decide *when a handler may run*, and we call them the *interrupt policy*. The remaining argument, restore_invariant, is not part of it. It writes into the state an invariant that the policy already maintains, for the benefit of the proof (Section V-F); it never changes a state the model reaches (Theorem 4), and it is the identity where the proof needs no help. Section IV fills the two parts of the interrupt policy twice.

## IV. Two Interrupt Policies

We now instantiate the model with two interrupt policies. The first, model_immediate, handles interrupts the ordinary way: a handler starts as soon as its interrupt can be serviced. It leaks (Section IV-A). The second, model_sliced, runs handlers only inside time slices opened by the public timer, each slice holding a fixed number of handler runs, with unused runs filled by the NOP handler. It does not leak (Section IV-B). Figure 5 runs both on the same inputs.

Both are exercised on the same concrete system. Its public user process emits the request GetRequest (Get) whenever it runs, and its private user process emits Syscall (Sys) when the disk handler has notified it and Idle otherwise. The scheduler alternates between the two user processes. Handlers take $runtime = 2$ steps, and the sliced policy uses slices of $runs = 2$ handler runs.

### A. Ordinary interrupt handling, and the leak

model_immediate is model at

```
immediate_init :=
  ((initial_pid, None),
   (false, (None, false_ic)))

immediate_service o v :=
  if is_ir_out_done o is Some ir then
    let v := unset_masks v in
    if ir is timer_irq then update_re_sched v true
    else v
  else v
```

and the identity for restore_invariant. In immediate_init (fields as in Section III-C) the slice counter is None, disabled, and false_ic clears every pending and mask bit. is_ir_out_done names the handler, if any, that emits Notify in the output o; the service policy then unmasks everything, and after the timer handler also requests a reschedule. An interrupt is therefore serviced at the first output step after it arrives at which no handler is running, and nothing constrains *when* that is. Section III-D traced the consequence: the disk handler starts at once and takes the public user's steps, producing the gap of Fig. 1.

**Theorem 2** (model_immediate_not_NI)**.** *For the concrete system,*

$$\mathsf{model_immediate_concrete} \notin \mathrm{NI}(\mathsf{in_equiv}, \mathsf{out_equiv}).$$

*Proof sketch.* Start from the trace of two public requests, which the model admits. Insert a disk interrupt at the front; it is unobservable at L, so non-interference demands that the result still be a trace. It is not. The interrupt sets the disk pending bit. The pid of the first step was fixed before any input, so the first request still goes through; but the next step belongs to the disk handler, so the public slot carries None where the second request was. □

A single counterexample refutes non-interference, so ordinary interrupt handling is not non-interfering in general.

### B. Sliced interrupt handling

The idea is to take away a secret interrupt's power to choose when its handler runs. Time is cut into slices, and a slice is opened only by the timer handler finishing, which is public. Each slice holds exactly *runs* handler runs (for handlers of at least two steps; Section V-F discusses one-step handlers), and a run that no disk interrupt claims is taken by the NOP handler.

model_sliced is model at sliced_init, sliced_service and restore_invariant:

```
sliced_init :=
  ((initial_pid, None),
   (false, (Some 0, mask_most)))

sliced_service runtime runs o :=
  check_ir_count
  o check_handler_completed runtime
  o initiate_ir runtime runs o

initiate_ir runtime runs o v :=
  if tI_out o is Some Notify then
    update_re_sched
      (update_ir_count v
         (Some (time_slice runtime runs)))
      true
  else v

check_handler_completed runtime v :=
  if handler_completed runtime
       (get_ir_count v) then
    update_ir_pending (set_tI (unset_masks v))
      nop_irq true
  else v

check_ir_count v :=
  match get_ir_count v with
  | Some n.+1 => update_ir_count v (Some n)
  | Some 0 => update_ir_count
                (set_otherIs (unset_tI v)) None
  | None => v
  end
```

The system starts outside a slice: mask_most masks the disk and NOP handlers and leaves the timer unmasked. The

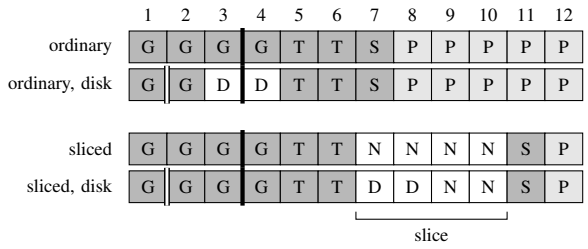



Fig. 5. Who runs at each step, under both policies, with and without a disk interrupt, on the concrete system of this section. G: public user, P: private user, S: scheduler, T, D, N: timer, disk and NOP handler; shaded as in Fig. 4. A solid bar marks the timer interrupt, a hollow bar the disk interrupt. Under the ordinary policy the disk handler takes steps 3 and 4 from the public user; under the sliced policy it takes the place of the first NOP run, and the shading of the two runs agrees. All four runs are proved traces of the mechanised models (`models.v`, Section 11).

slice counter `ir_count` counts output steps, and `sliced_service` runs three stages on each output, right to left:

- *Opening a slice.* When the timer handler emits `Notify`, `initiate_ir` loads the counter with time_slice $runtime\ runs = runs \cdot runtime$, the number of steps that *runs* handler runs take, and requests a reschedule for when the slice ends.
- *Boundaries.* `handler_completed` holds when the counter is a nonzero multiple of *runtime*: at the start of the slice and after each completed run inside it. There `check_handler_completed` reopens the masks, masks the timer again, and makes the NOP handler pending. That last step is the *filler*: a handler is always ready at a boundary, so a run takes place whether or not a disk interrupt is waiting, and the disk handler, if pending, takes priority.
- *Closing.* `check_ir_count` ticks the counter, and when it reaches zero masks the disk and NOP handlers, unmasks the timer and disables the counter.

On the step where the timer handler completes, opening a slice and its first boundary coincide, so the first run of the slice starts at the very next step. Outside a slice a disk interrupt is only recorded as pending; inside one, the timer is masked, so it cannot start a handler mid-slice. When a slice ends, the reschedule requested at its opening hands control to the scheduler.

A secret handler run therefore takes the place of a NOP run: a slice containing a disk run has the same length and position as one containing only filler, and the public schedule around it is unchanged. Table II shows this on the concrete system. The two runs differ only in a disk interrupt; it is recorded as pending, waits for the next slice, and then replaces the first NOP run. Only two rows differ: one in which handler ran, and one in the private user's value. By Fig. 4, the attacker sees neither, so the two runs look the same to it. Theorem 3 in Section V proves that the same holds for any two runs that differ only in inputs the attacker cannot see.

## V. The Security Proof

**Theorem 3** (`model_sliced_NI`)**.** *If the scheduler and the public user process are non-interfering at* `public_equiv` *on input and output, and the private user process is non-interfering at* `private_equiv` *on input and output, then*

$$\mathsf{model_sliced}\ runtime\ runs\ p_{pub}\ p_{priv}\ p_{sched} \in \mathrm{NI}(\mathsf{in_equiv}, \mathsf{out_equiv}).$$

**Corollary 1** (`model_sliced_userview_NI`)**.** *Under the same hypotheses,* `model_sliced_userview` *is non-interfering at* `out_equiv_userview`*.*

The corollary follows from Theorem 3 by `map_NI`, since `parse_output` maps `out_equiv`-related outputs to `out_equiv_userview`-related ones. The rest of this section is about Theorem 3.

### A. The equivalences

`in_equiv` makes the disk interrupt, and only the disk interrupt, unobservable at L. The timer stays public, because the sliced policy schedules by it.

A pool output is a six-tuple of options, one per slot, and `out_equiv` is the corresponding nest of `eqpair`s (Table III). Its two columns come apart only for the two secret handlers, and that is where scheduling secrecy lives. The payload equivalence hides a slot's *value*. Whether the slot *ran* is decided by the `eqmaybe` instance around it, which fixes who can see `None`. Under `eqmaybe_all` the attacker tells `Some` from `None`, and so sees that the private user process ran; under `eqmaybe_nonbot` a handler run and a step in which nothing happened are the same observation.

The NOP slot must be classified like the disk slot: a slice run is filled by one or the other, so the two must look alike.

### B. Compositional structure

The proof follows the term of `model_sliced` from the outside in, applying one composition theorem per constructor (Theorem 1): `map_NI` for the outer rewiring, `loop_NI` for the feedback, `sta_NI` for the global state, `maybe_NI` for the idling pool, `par_NI` to split the pool into slots, `swi_NI` for each slot's gate, and at the leaves `out_NI` for the handler processes and the hypotheses of Theorem 3 for the scheduler and user processes. Since each theorem derives the composite's equivalences from those of its parts, the shape of `out_equiv` is forced: `par_NI` imposes its nest of `eqpair`s and `swi_NI` its `eqmaybe` instances.

Two of these steps carry substantial side conditions: the gate obligations of `swi_NI` (Section V-C) and the state obligations of `sta_NI` (Sections V-D and V-E).

### C. Gating a slot

Each slot is a `swi` whose gate bit is computed from the pid by a `map`. Two obligations remain per slot, both parameterised by an equivalence $\dot{=}_B$ on the gate bit that the proof may choose. `map_NI` requires the gate map to be unobservable-preserving,

TABLE II

TWO RUNS OF THE CONCRETE SLICED SYSTEM THAT DIFFER ONLY IN A DISK INTERRUPT. EACH ROW IS ONE OR MORE CONSECUTIVE OUTPUT STEPS OF ONE PROCESS; THE ROWS IN ITALICS ARE INPUTS. A HANDLER STEP IS SHOWN AS THE HANDLER AND ITS OUTPUT (NOTHING OR NOTIFY).

| without disk interrupt | with disk interrupt | |
|---|---|---|
| `Get` | `Get` | |
| | *disk interrupt* | recorded as pending; the disk handler is masked |
| `Get`, `Get` | `Get`, `Get` | |
| *timer interrupt* | *timer interrupt* | |
| `Get` | `Get` | |
| timer: `Nothing`, `Notify` | timer: `Nothing`, `Notify` | the timer handler completes: a slice opens |
| NOP: `Nothing`, `Notify` | disk: `Nothing`, `Notify` | the disk handler replaces the first NOP run |
| NOP: `Nothing`, `Notify` | NOP: `Nothing`, `Notify` | second run of the slice |
| scheduler: private user | scheduler: private user | the slice is over: the scheduler runs |
| `Idle` | `Sys` | the private user reacts; its value is secret |

TABLE III

OUT_EQUIV, ONE COMPONENT PER SLOT: A WRAPPER FROM THE EQMAYBE FAMILY AROUND A PUBLIC OR PRIVATE PAYLOAD EQUIVALENCE. THE LAST TWO COLUMNS SAY WHAT THE ATTACKER AT L SEES.

| slot | wrapper | payload | value | ran? |
|---|---|---|---|---|
| public user | `eqmaybe_all` | public | visible | visible |
| private user | `eqmaybe_all` | private | hidden | visible |
| scheduler | `eqmaybe_all` | public | visible | visible |
| NOP | `eqmaybe_nonbot` | private | hidden | hidden |
| disk | `eqmaybe_nonbot` | private | hidden | hidden |
| timer | `eqmaybe_all` | public | visible | visible |

TABLE IV

STATE_TYPE_EQUIV, FIELD BY FIELD.

| field | equivalence | at L |
|---|---|---|
| current pid | `eqsum private_equiv public_equiv` | which secret handler is hidden |
| previous pid, `re_sched`, `ir_count` | `public_equiv` | visible |
| NOP and disk bits | `private_pending_equiv` | pending hidden, mask visible |
| timer bits | `public_bits_equiv` | visible |

sending unobservable pids to unobservable bits (`f_PU`), and `swi_NI` requires, at every level, that the observer be aware of the gate or oblivious to the slot:

$$\ell \in \mathsf{A}(\mathsf{true}, \overset{B}{\doteq}) \ \cup \ \mathsf{O}(\mathsf{swi}\ b\ p,\ \mathsf{eqmaybe_aware}(\overset{O}{\doteq}, \overset{B}{\doteq})).$$

Awareness means a bit related to `true` is `true` and observable, so the observer can trust the gate. Obliviousness means the observer sees none of the slot's outputs, whatever the input does.

At L the two pids of the secret handlers are unobservable. For a slot neither of them selects, both map to `false`, so a $\overset{B}{\doteq}$ in which `false` alone is unobservable satisfies `f_PU`; we call it `false_equiv`. Under `false_equiv` the observer is aware at every level, and the obligation is discharged without a case split. This covers the public and private user slots, the scheduler and the timer handler.

For the disk and NOP slots, one secret pid maps to `true` and the other to `false`, so `f_PU` demands that both bits be unobservable, which leaves only `private_equiv`. But then `true` is unobservable at L and awareness fails there. At H awareness still holds; at L the proof uses obliviousness. A step where the slot is gated off emits `None`, and a step where it runs emits `Some o`, so both must be unobservable at L. Under `eqmaybe_aware`, `None` is unobservable exactly when the gate is hidden, and `Some o` when the payload is, which the `private_equiv` payload of Table III provides. The line therefore falls between the two secret handlers and everything else; the private user process sits on the public side, its secrecy carried by its payload and never by its gate.

### D. The state equivalence

`sta_NI` imposes two conditions on the state update, and they pull in opposite directions.

- `f_EP` requires that an unobservable input not visibly move the state. A disk interrupt is unobservable at L and sets the disk pending bit, so that bit must be private. (The event equivalence `eqsum_L in_equiv out_equiv` makes only interrupts unobservable, so `f_EP` constrains `set_pending` alone.)
- `fv_NI` requires that related states be mapped to related states. `initiate_next` branches on the mask bits, so two related states that disagreed there would take different branches; the mask bits must be public.

Both hold only if the interrupt controller is classified per bit: the disk and NOP pending bits are private and every mask bit is public (Table IV). The same constraint explains why `set_pending` sets one bit and stops: any decision taken on an input step, such as starting a handler or moving the slice counter, would disturb a public field. Everything an interrupt causes is deferred to a later output step, where the input is no longer the thing being varied.

The `inl` side of the pid type holds exactly the two secret handlers (Section III-C), which lets the proof classify a whole pid in one place.

The current pid is where the policy does its work. Since `eqsum` never relates an `inl` to an `inr`, the fact *that* a

handler is running stays visible, as does every public pid. What private_equiv hides is the bit inside inl: at L the disk handler and the NOP handler look alike. Across two L-related states, that bit and the two private pending bits are the only things allowed to differ.

### E. Closure under the transition

The core obligation is that state_type_equiv is closed under the whole transition. The transition is a sum, set_pending on interrupts and a pipeline of stages on outputs, and the event equivalence never relates an interrupt to an output. So the goal splits along the sum, and a composition lemma (fv_NI_comp) breaks the pipeline into one goal per stage:

- set_pending: related interrupts set related pending bits;
- check_scheduler: the scheduler's pid is public, so the update agrees;
- service_policy: the slice logic reads only the public counter and masks, and writes the same values into both states;
- initiate_next: related states must make the same choice of what runs next.

Each goal is self-contained, which is the point of splitting. The cost is that a stage must be proved for *all* pairs of related states, since it cannot assume its input came from the stage before. That loses the invariant of the time slice, and the last stage cannot be proved without it.

### F. Restoring the invariant

Two facts hold of every state model_sliced reaches. While a slice is live and no handler is running, the NOP handler is pending, the disk and NOP masks are clear and the timer is masked; once the slice is over, the disk and NOP handlers are masked. And the disk and NOP masks always agree. The sliced policy maintains both (Section IV-B).

The function restore_invariant writes these facts into the state, just before initiate_next chooses. It reads only the public counter, so it sets the same bits in two related states, and afterwards the deciding mask bits agree and related states take the same branch. This is what discharges the initiate_next goal.

We prove that it never changes a state the model reaches.

**Theorem 4** (restore_invariant_noop)**.** *Let* $runtime \neq 1$*. For every sequence of events* $es$ *from the initial state of* model_sliced*, and every output* $o$*, let* $w$ *be the state that* initiate_next *receives on* $o$ *after* $es$*. If not all masks are set in* $w$*, then* restore_invariant $w = w$.

The events $es$ are arbitrary, so the theorem holds whatever the pool emits. The proof states the invariant between steps, shows that every event preserves it, and shows that at the choice point a slightly weaker invariant holds, on which restore_invariant is the identity. One small lemma per stage of the pipeline does the bookkeeping.

TABLE V
WHAT THEOREM 3 LEAVES OPEN.

| parameter | ranges over | side condition |
|---|---|---|
| p_pub | any public user process | NI at public_equiv |
| p_priv | any private user process | NI at private_equiv |
| p_sched | any scheduler | NI at public_equiv |
| Opub, Opriv | the user processes' output alphabets | none |
| runtime | handler length | none |
| runs | handler runs per slice | none |

The hypothesis $runtime \neq 1$ exposes a quirk of the model. The boundary test reads the counter before the stage that ticks it, while restore_invariant reads it after. With one-step handlers the last boundary inside a slice ticks the counter to zero in the same step, so the slice already reads as over, and restore_invariant masks the handler that boundary had just enabled. Non-interference is unaffected (Theorem 3 has no such hypothesis), but at $runtime = 1$ every slice loses its last handler run.

An earlier version of the model relied on restore_invariant for behaviour: it was the only thing that queued the NOP filler and the only thing that returned control to the scheduler after a slice. Proving Theorem 4 required moving both into service_policy, where they now appear as part of the policy (Section IV-B). The proof of Theorem 3 needed only two small lemmas to follow the change.

## VI. HOW GENERAL THE RESULT IS

Theorem 3 covers a family of systems. Discharging the section variables of the mechanisation, it reads in full:

```
forall (Opub Opriv : Ty) (runtime runs : nat)
       (p_pub : Proc Empty Opub)
       (p_priv : Proc THandlerOutput Opriv)
       (p_sched : Proc Empty Nat),
  NI (public_equiv Empty) (public_equiv Opub)
     p_pub ->
  NI (private_equiv THandlerOutput)
     (private_equiv Opriv) p_priv ->
  NI (public_equiv Empty) (public_equiv Nat)
     p_sched ->
  NI in_equiv (out_equiv Opub Opriv)
     (model_sliced runtime runs
                   p_pub p_priv p_sched)
```

Sections III–V fixed the output types of the two user processes to those of the example. The theorem is parametric in them (Opub and Opriv above). Everything the pool is built around stays fixed; everything it carries is left open (Table V). The side conditions are the expected ones: the mechanism cannot make a leaking process secure, so each slot is assumed non-interfering at its own classification, and the theorem says that composing them through this mechanism leaks nothing further. The handler length and slice size carry no side condition, because the slice is *defined* as $runs \cdot runtime$ and so ends on a handler boundary by construction.

Two features of the model make this possible. The state update never reads the *value* a user slot emits, only whether it emitted (the scheduler-output test inspects the user slots for

TABLE VI
THE DEVELOPMENT.

| file | lines | contents |
|---|---|---|
| definitions.v | 908 | the calculus, traces, NI, the equivalences |
| theorems.v | 1337 | the composition theorems |
| models.v | 1079 | the generic model, both policies, Theorem 4, example traces |
| noninterference.v | 1749 | the model's equivalences, Theorems 2 and 3 |
| adequacy.v | 904 | the original definitions, the equivalences with ours, Theorems 2 and 3 in the original's terms |

None and nothing more), so no user behaviour reaches the schedule. And the state obligation fv_NI never mentions the slot processes, so the one hard obligation is unaffected by what fills them.

What stays fixed is the mechanism under study, the three handlers, and the shape of the pool around it: six slots in a fixed order. The order is hard-wired in the projections that read the pool's output, and varying the number of user slots would change the state transition and with it the proof of fv_NI.

*a) Levels:* We have read levels as two. The mechanisation works over an arbitrary bounded lattice, where a characterised equivalence must also be monotone downwards (what a level relates stays related below it, and likewise for unobservability), and NI quantifies over every level. Every equivalence in the models treats the bottom level, read as $\mathsf{L}$, apart from all others, so the proof's level case splits are $\ell = \mathsf{L}$ against $\ell \neq \mathsf{L}$, and the second case covers every other level of any lattice at once.

## VII. Mechanisation

The development is about 6,000 lines of Rocq 9.0.1, available at https://github.com/Tirore96/NonInterference, using Mathematical Components [5] and, for the coinductive definitions of the original paper, paco [6] (Table VI).

*a) Types:* Input and output types range over an inductive universe Ty of codes, interpreted into Set, rather than over Set directly. This makes reductions invertible, which the negative result needs: refuting non-interference means showing that a trace is *not* admitted.

*b) Axioms:* The development assumes nothing beyond the two parameters that fix the security lattice: no classical logic, and no uniqueness of identity proofs. The latter is what dependent elimination on the type-indexed reduction relations would need; we invert reductions through determinism of reduction instead, one lemma per rule (inv_mapO and its siblings).

### A. Non-interference over finite traces

Our definitions follow Rafnsson et al., with one essential departure. The original defines non-interference as a coinductive simulation over infinite streams of labels, whose clauses apply at the head of a stream. We quantify over finite traces instead, with closure clauses that apply at an arbitrary position (Definition 1). This makes every proof in the development an induction on a trace.

The original definition is written out in adequacy.v, and NI_paper proves the two equivalent. The proof passes through infinite streams (NI_SNI) and rests on two properties: finite and infinite behaviour determine each other, which needs reduction to be deterministic (to assemble finite runs into one infinite run) and never to block (to continue a finite run); and insertion commutes with taking prefixes. Obliviousness, the other definition that quantifies over behaviour, is likewise stated over traces and equivalent to the original (ObliviousDis_iff). As a consequence, Theorems 2 and 3 also hold with non-interference read as in the original, hypotheses included (model_immediate_not_NI_paper, model_sliced_NI_paper).

One further difference is only notational. The original adjoins the distinguished value $\bullet$ to every type; we keep the values as they are and make unobservability a predicate, tied to the equivalence by the characterisation law. NI_lEquiv shows that the two give the same notion of non-interference, and this paper uses the original notation throughout.

The composition theorems themselves are proved directly for our definitions.

## VIII. Related Work

*a) Non-interference:* Non-interference goes back to Goguen and Meseguer [2]; Sabelfeld and Myers survey language-based approaches [1]. Timing channels in programs have been closed by padding transformations that equalise the running time of secret-dependent branches [3]; the sliced policy applies the same idea to handler runs, with the NOP handler as padding.

*b) Composable timing-sensitive non-interference:* Our proofs rest on the calculus of Rafnsson et al. [4], whose definitions are timing-sensitive by construction and whose theorems compose. We mechanise the calculus and its composition theorems, and apply them to a model of interrupt handling.

*c) Verified operating systems:* Machine-checked information-flow proofs exist for real kernels, notably seL4 [7], whose proof explicitly leaves timing channels out of scope. Time protection [8] argues that an operating system must partition time as well as space, and includes the partitioning of interrupts among its mechanisms[TODO: check this characterisation against [8]]; our sliced policy is an instance of that idea in a small model. What we add is a timing-sensitive, compositional, mechanised proof that the mechanism is secure, for any non-interfering scheduler and user processes.

## IX. Limitations and Conclusion

*a) Limitations:* The model makes several simplifications.

- *One secret interrupt.* The NOP handler exists only as the disk handler's indistinguishable partner. Several secret interrupts would need a richer indistinguishability argument.

- *Equal handler lengths.* Every handler runs for the same number of steps, which is what lets every mask bit be public. Variable-length handlers would force the masks private, and the proof of `fv_NI` would have to reason across diverging branches. A real system would reach a fixed length by padding.
- *Cooperative processes, no nesting.* A selected process takes exactly one step; preemption within a step is not modelled. Handlers cannot be interrupted, since all masks are set while one runs.
- *Abstract time.* One output step is one unit of time, for every process and handler alike. Caches, memory contention and other microarchitectural channels are outside the model.
- *One-step handlers.* At $runtime = 1$ a slice loses its last handler run (Section V-F); the result still holds, but the policy behaves less usefully there.

*b) Conclusion:* A secret interrupt can leak through the schedule alone, and servicing interrupts only in publicly timed, padded slices closes the channel. We proved both facts in Rocq, the second for any non-interfering scheduler and user processes, by composing generic non-interference theorems over the structure of the model. The proof did more than check the policy: it fixed the classification of the interrupt controller bit by bit, separated hiding what a handler computed from hiding that it ran, and exposed a proof device that was quietly carrying behaviour, which we moved into the policy and then proved inert.

## APPENDIX

The generic model puts the pool inside a stateful wrapper, `reactive_system`:

```
model runtime init_policy service_policy
      restore_invariant p_pub p_priv p_sched =
  reactive_system init_policy
    (state_step service_policy restore_invariant)
    no_output
    (pool runtime p_pub p_priv p_sched)
    pool_input
  : Proc T_in T_out
```

`no_output`, the all-`None` tuple, stands in for the pool's output before its first step, and `pool_input` passes each slot the current pid and the disk handler's output.

*a) Handlers:* `ir_handler runtime` counts its output steps modulo `runtime`. Its output type `THandlerOutput` has two values: it emits `Notify` on the step that completes a run and `Nothing` (a handler value, not the option `None`) otherwise.

*b) The pool:* Each slot is a `swi` whose gate bit is computed from the pid. Each slot's output is tagged with the constant `true`, which closes its switch after one output step, so a selected process takes exactly one step.

*c) The scheduling decision:* The priority rule of Section III-D is

```
initiate_next restore_invariant v =
  if masks_set v then v    (* handler running *)
  else let v := restore_invariant v in
  if first_ready v is Some ir
  then initiate_handler ir v
  else if is_handler_pid v (* handler done *)
  then if get_re_sched v
       then initiate_scheduler v
       else initiate_prev_pid v
  else v                   (* stay in user space *)
```

`first_ready` picks the highest-priority serviceable interrupt. `initiate_handler` saves the current user pid, points the current pid at the handler, sets all masks and clears the handler's pending bit. `initiate_scheduler` hands control to the scheduler and clears `re_sched`; `initiate_prev_pid` returns to the interrupted user process.

## REFERENCES


[1] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.

[2] J. A. Goguen and J. Meseguer, "Security policies and security models," in *IEEE Symposium on Security and Privacy*, 1982, pp. 11–20.

[3] J. Agat, "Transforming out timing leaks," in *Principles of Programming Languages (POPL)*. ACM, 2000, pp. 40–53.

[4] W. Rafnsson, L. Jia, and L. Bauer, "Timing-sensitive noninterference through composition," in *Principles of Security and Trust (POST)*, ser. Lecture Notes in Computer Science, vol. 10204. Springer, 2017, pp. 3–25.

[5] A. Mahboubi and E. Tassi, *Mathematical Components*. Zenodo, 2022.

[6] C.-K. Hur, G. Neis, D. Dreyer, and V. Vafeiadis, "The power of parameterization in coinductive proof," in *Principles of Programming Languages (POPL)*. ACM, 2013, pp. 193–206.

[7] T. Murray, D. Matichuk, M. Brassil, P. Gammie, T. Bourke, S. Seefried, C. Lewis, X. Gao, and G. Klein, "seL4: From general purpose to a proof of information flow enforcement," in *IEEE Symposium on Security and Privacy*, 2013, pp. 415–429.

[8] Q. Ge, Y. Yarom, T. Chothia, and G. Heiser, "Time protection: The missing OS abstraction," in *European Conference on Computer Systems (EuroSys)*. ACM, 2019.